Assignment 2

Comparative Environmental Impact of Model Compression

Quantization, pruning and knowledge distillation compared on CO_2 emissions, latency, model size and accuracy, measured with CodeCarbon

Submitted by	Nishad Ahamed
Course	ENSF 619
Date	October 2026
Code	github.com/ahmd-nish/ensf619-assignments (folder Assignment_2)

Table of Contents

1. Summary Table

2. Implementation Details

2.1 Baseline model

2.2 Quantization

2.3 Pruning

2.4 Knowledge distillation

2.5 Training data

2.6 CodeCarbon setup

3. Comparative Analysis

3.1 Which technique achieved the lowest CO_2 emissions?

3.2 How does model size relate to CO_2 emissions?

3.3 Which technique gave the best environmental-performance trade-off?

3.4 How difficult was each technique to implement?

Appendix: Files and Reproduction

1. Summary Table

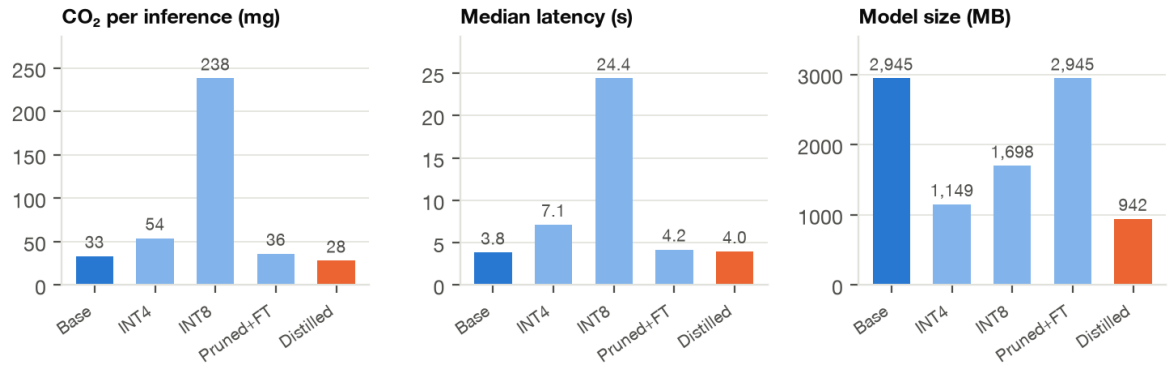
I compared four versions of the same small language model on the Assignment 1 task: writing a Google-style docstring for `below_zero`. I used prompt P-2, the best prompt from Assignment 1. Each version ran **5 times**. For every run, a new **CodeCarbon** tracker was started right before generation and stopped right after it.

Model version	Technique	CO_2 per inference (mg CO_2eq)	vs. baseline	Median latency (s)	Model size (MB)	vs. baseline	Accuracy (P/F)
1. Baseline (FP16)	None	33.3	—	3.84	2,945	—	**P** (5/5)
2. Quantized (4-bit NF4)	Quantization	53.9	+62%	7.12	1,149	–61%	**P** (5/5)
2b. Quantized (8-bit), extra	Quantization	238.0	+615%	24.39	1,698	–42%	**P** (5/5)
3. Pruned (40% + fine-tune)	Pruning	36.0	+8%	4.20	2,945*	0%	**P** (5/5)
4. Distilled (0.5B student)	Distillation	**28.0**	**–16%**	3.97	**942**	**–68%**	**P** (5/5)

CO_2 is the mean of 5 runs. The variation between runs was small (SD 0.3–2.4 mg), except for 8-bit (SD 104 mg). Model size is the saved `state_dict`, measured with the function given in the assignment. *The pruned model is still stored as a full-size file, because the zeros take up space. Stored in a sparse format it would be about 2,100 MB (–29%).

Test setup:

- **Hardware:** NVIDIA RTX A6000 (48 GB) on a rented Vast.ai machine in Bucharest, Romania. CodeCarbon used the local grid intensity of 240.6 g CO_2/kWh.
- **Same conditions for every model:** greedy decoding, up to 512 new tokens, and one warm-up run that was not counted.
- **Accuracy (P/F):** the same check as Assignment 1. Every compressed model still produced a correct, well-formatted docstring in all 5 runs.
- **Extra check:** each model also wrote docstrings for 33 functions it had never seen, to test accuracy more broadly.



2. Implementation Details

2.1 Baseline model

Qwen2.5-Coder-1.5B-Instruct, with 1.54 billion parameters, run in FP16. I chose it because:

- it was the most accurate model in Assignment 1, and
- it has a smaller sibling model with the same tokenizer, which makes distillation straightforward.

The baseline also acted as the **teacher** for distillation.

2.2 Quantization

- **Tool:** the bitsandbytes library, through `BitsAndBytesConfig` in Hugging Face Transformers.
- **Main version: 4-bit NF4** (`load_in_4bit=True`, `bnb_4bit_quant_type="nf4"`), with calculations done in FP16.
- **Extra version: 8-bit** (LLM.int8), included because Assignment 1 also tested it.
- **No training needed.** The weights are compressed when the model loads.

2.3 Pruning

- **Method:** unstructured L1 (magnitude) pruning with `torch.nn.utils.prune.l1_unstructured`.
- **Ratio: 40%.** In each of the 196 linear layers inside the 28 transformer blocks, the 40% of weights with the smallest values were set to zero. The embedding layer was left untouched. In total, 34% of all parameters became zero.
- **Recovery fine-tune:** 2 epochs (224 steps), AdamW, learning rate 1e-5, batch size 8. After every step the zeroed weights were set back to zero, so the sparsity stayed at exactly 40%.

2.4 Knowledge distillation

- **Teacher:** the baseline Qwen2.5-Coder-1.5B, frozen.
- **Student: Qwen2.5-Coder-0.5B-Instruct**, with 0.49 billion parameters, 3.1 times smaller. It uses the same 151,936-token vocabulary as the teacher.
- **Loss:** half “match the teacher’s probabilities” (KL divergence, temperature 2) and half “match the teacher’s written answer” (cross-entropy), so $\alpha = 0.5$.
- **Training:** 3 epochs (336 steps), AdamW, learning rate 2e-5, batch size 8.

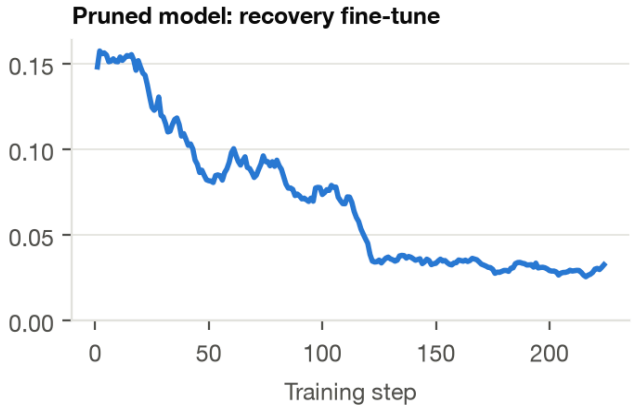
2.5 Training data

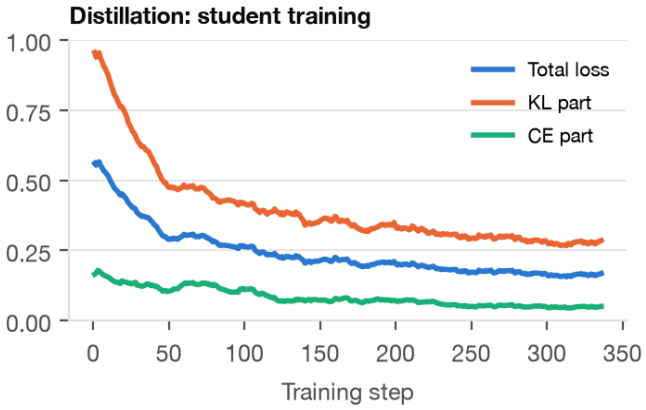
Pruning and distillation used the same training data:

- **Source:** the teacher wrote docstrings for 1,364 Python functions from the HumanEval and MBPP datasets.
- **Filtering:** only answers that passed the accuracy check were kept, leaving **892**.
- **Kept out of training:** the `below_zero` function itself (HumanEval/3) and the 33 test functions were never used for training.

2.6 CodeCarbon setup

- **Tracker:** CodeCarbon 3.3.2, `EmissionsTracker(measure_power_secs=1, tracking_mode="process")`, started and stopped around each `generate()` call.
- **GPU power** was measured directly through NVIDIA's NVML interface, and it made up about 85% of the energy.
- **CPU and RAM power** were estimated by CodeCarbon (about 8 W and 10 W), because the rented machine does not expose CPU power sensors.





3. Comparative Analysis

3.1 Which technique achieved the lowest CO_2 emissions?

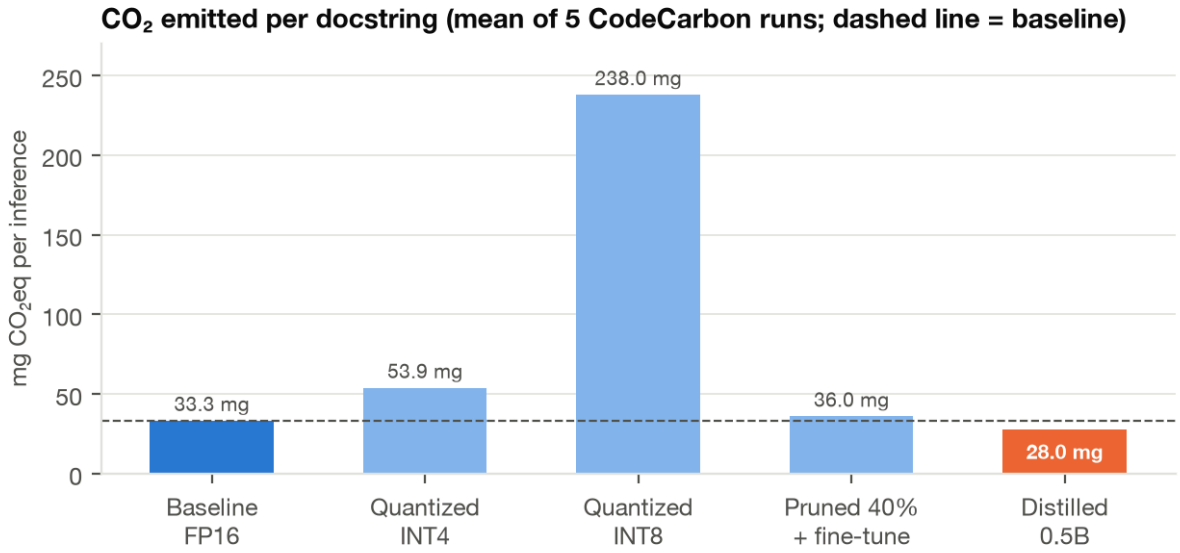
> **Knowledge distillation**: 28.0 mg CO_2eq per inference, **16% lower** than the baseline (33.3 mg).

The distilled 0.5B model was the only compressed version that emitted less than the original. It produced slightly longer answers (151 tokens against 129), but each token was much cheaper: 2.8 J against 3.9 J for the baseline.

The other techniques ranked:

- **Pruning:** 36.0 mg, 8% above the baseline.
- **4-bit quantization:** 53.9 mg, 62% above.
- **8-bit quantization:** 238 mg, more than 7 times the baseline.

The 0.5B student *before* distillation emitted even less (20.3 mg), but it failed the task. A model that gives the wrong answer is not a green option.



3.2 How does model size relate to CO_2 emissions?

> **A smaller model did not automatically mean lower emissions.** Only distillation reduced both size and CO_2.

Quantization made the model smaller but emitted more.

- 4-bit cut the size by 61% but increased CO_2 by 62%.
- 8-bit cut the size by 42% but increased CO_2 more than 7 times.

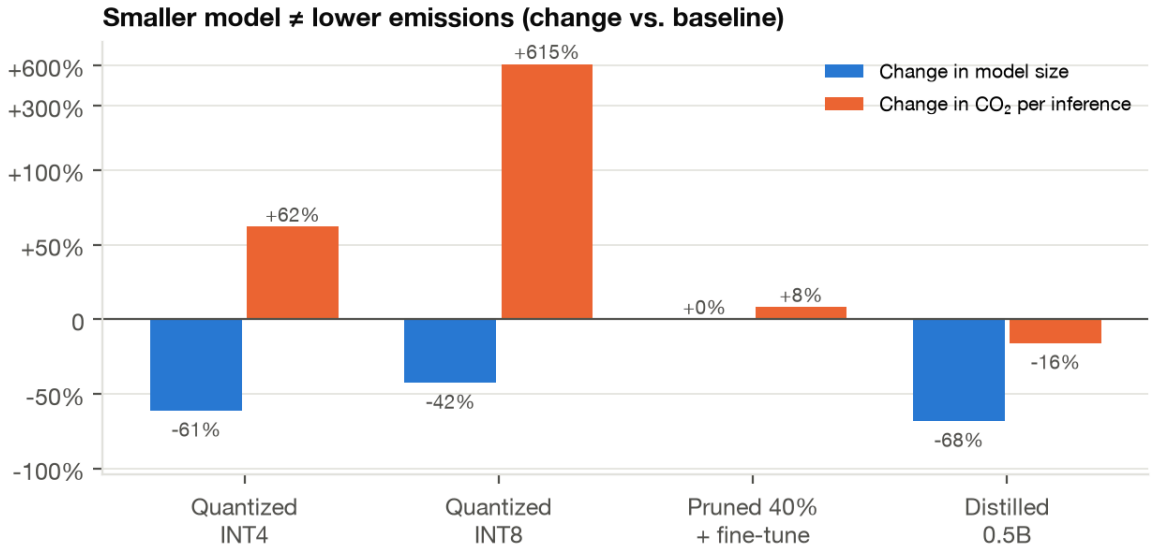
The reason is that bitsandbytes stores the weights compactly but has to unpack them during every calculation. That extra work outweighs any saving. Our model is small and the GPU is large, so memory was never the bottleneck to begin with.

Pruning made no difference to size or speed. 34% of the weights became zero, but the zeros are still stored and still multiplied by the GPU. Standard PyTorch has no way to skip them. CO_2 even rose by 8%, because the pruned model wrote 9 more tokens.

Distillation reduced the real work. The student has 3 times fewer parameters and fewer layers (24 against 28), so each token takes less computation. Size fell by 68% and CO_2 by 16%.

CO_2 fell much less than size because, for one request at a time, a fixed share of the time goes on overhead that model size does not change.

Takeaway: emissions follow the amount of computation that actually runs on the hardware, not the file size on disk. Compression only saves energy if the hardware and software can take advantage of it.



3.3 Which technique gave the best environmental-performance trade-off?

> **Knowledge distillation**: the lowest CO_2 of any version that still passed, a 68% smaller model, the same speed, and 5/5 correct answers.

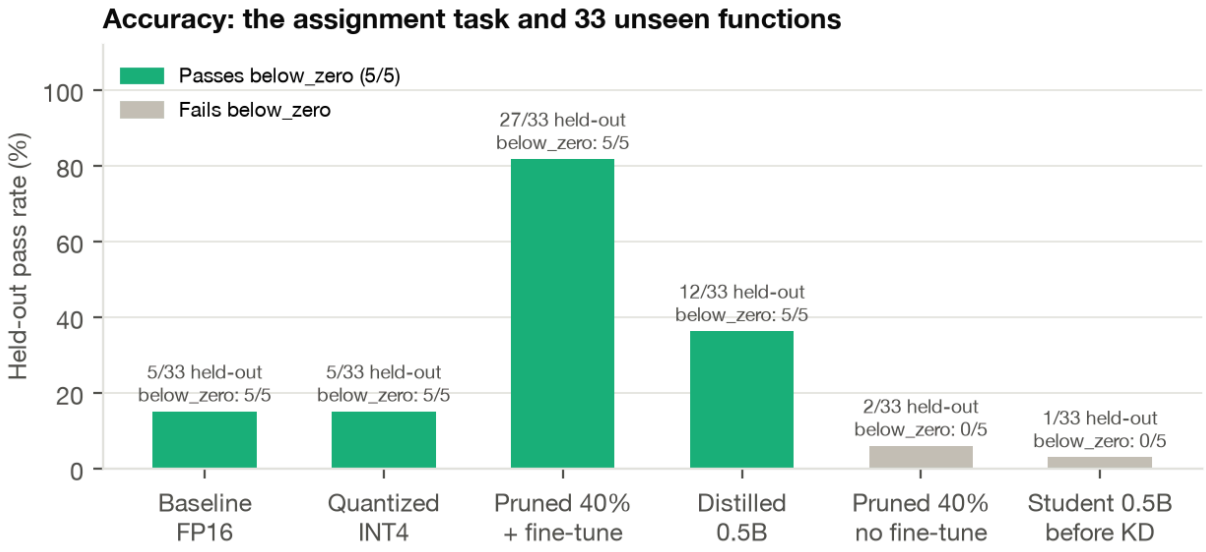
Why distillation wins:

- **Lowest emissions while accurate:** 28.0 mg against 33.3 mg for the baseline.
- **Smallest file:** 942 MB against 2,945 MB.
- **No slower:** 3.97 s against 3.84 s.
- **Accurate and consistent:** correct in all 5 runs, with identical output each time.

Training was what made the compressed models work. The two comparison models in the chart below show this:

- The **0.5B student before distillation** failed the task (0/5). Its docstring had no Args or Returns sections.
- The **pruned model before fine-tuning** also failed (0/5), and it rambled: 217 tokens, with 67% more CO_2 than the baseline.

A note on the extra 33-function check: the pruned (27/33) and distilled (12/33) models scored higher than the baseline (5/33). That is because they were trained only on correctly formatted examples, which also taught them the required docstring style. The gain comes from that training, not from compression itself. Without training, both compressed models did worse than the baseline (2/33 and 1/33).



3.4 How difficult was each technique to implement?

> **Quantization was the easiest, pruning was moderate, and distillation was the hardest.**
> Distillation was also the only one that paid off in emissions.

	Quantization	Pruning	Distillation
Code needed	One setting when loading the model	A few lines to prune, plus a training loop	A full training loop with a custom loss
Training data	None	Needed (892 examples)	Needed (892 teacher answers)
Training time	None	About 7 min on the A6000	About 5 min on the A6000
GPU memory	Low	High: full fine-tuning of 1.5B (about 25 GB)	High: teacher and student in memory together
Main difficulty	Only works on NVIDIA GPUs (not on the Mac from Assignment 1)	Keeping the zeros at zero during training, and needing special kernels to get any real speed-up	Building and filtering the training data, and choosing α and the temperature
Emissions result	Worse (+62% / +615%)	Slightly worse (+8%)	**Better (–16%)**

Quantization was by far the easiest: one line of configuration, no data and no training. But it only works on NVIDIA GPUs, and here it made emissions worse.

Pruning is easy to apply but hard to make useful. Recovering accuracy needed labelled data and a full fine-tuning run. Getting a real speed-up would need sparse file formats or special hardware support, which plain PyTorch does not provide.

Distillation took the most work: a compatible student model, a training set generated by the teacher, a custom loss, and both models in GPU memory at once. It also has a one-time training cost, estimated at about 25 Wh or 6 g CO_2 (not measured). With a saving of 5.3 mg per inference, that cost is repaid after roughly 1,100 docstrings.

Overall: for small code models answering one request at a time, distillation is the best choice for reducing emissions. Use quantization only where fast low-bit support exists, and use pruning only together with tools that can skip the zero weights.

Appendix: Files and Reproduction

The code is at https://github.com/ahmd-nish/ensf619-assignments/tree/main/Assignment_2.

File	Contents
`results/summary.csv`, `results/results.xlsx`	Summary table and every CodeCarbon run
`results/runs.csv`	One row per run: CO_2, energy (GPU / CPU / RAM), latency, tokens, P/F
`results/codecarbon/emissions.csv`	The raw CodeCarbon output
`results/variant_info.json`	Model size, parameter counts and sample output for each model
`results/prune_log.json`, `results/distill_log.json`	Training settings and loss curves
`build_dataset.py` → `prune_finetune.py` → `distill.py` → `evaluate.py`	The pipeline, in order (`run_all.sh` runs all four)
`analyze.py`, `report_figures.py`	Summary tables and charts

To reproduce on a CUDA GPU:

```
pip install torch transformers accelerate bitsandbytes codecarbon datasets pandas openpyxl
matplotlib
./run_all.sh && python analyze.py && python report_figures.py
```